

**UNIVERSIDADE KIMPA VITA
INSTITUTO POLITÉCNICO
LICENCIATURA EM ENGENHARIA INFORMÁTICA**

RELATÓRIO TÉCNICO DE S.G.B.D

*Arquitectura e Modelação Avançada em Sistemas de Gestão de Bases de Dados
NoSQL “AngolaCommerce”*

GRUPO: 19

Por:

António Dias Guilherme
Elisa Lovola
Luís Fernando José
Pedro Igildo

DOCENTE:

Moyo Kanivengidio

UÍGE, 2026

RESUMO

O presente relatório descreve o desenvolvimento do projecto **AngolaCommerce Analytics**, uma plataforma académica de análise avançada de dados de comércio electrónico em Angola, construída sobre uma arquitectura NoSQL com MongoDB, no âmbito da disciplina de Linguagem de Programação VIII do 4.º ano de Engenharia Informática do IPUKV.

A base de dados foi populada com **10.000.009 documentos** distribuídos por 8 colecções (clientes, produtos, encomendas, avaliações, logs, pesquisas, notificações e carrinhos), num processo de geração massiva que durou **46,9 minutos**. O projecto inclui ainda uma dashboard analítica interactiva, consultas avançadas com o Aggregation Pipeline, criação estratégica de índices e uma API RESTful em Node.js/Express.

Os resultados demonstram que a arquitectura NoSQL com MongoDB é superior ao modelo relacional em cenários de alta volumetria, schema flexível e distribuição geográfica, com taxa média de inserção de **4.119 documentos/segundo** e tempos de consulta abaixo de 5 ms com indexação adequada.

Palavras-chave: *MongoDB, NoSQL, Big Data, Comércio Electrónico, Angola, Aggregation Pipeline, Indexação, Dashboard Analítica.*

1. INTRODUÇÃO

1.1 Contexto e Motivação

O crescimento exponencial do comércio electrónico em África, e em Angola em particular, tem gerado volumes massivos de dados transaccionais que os sistemas relacionais tradicionais encontram dificuldade em processar com eficiência. Limitações de escalabilidade horizontal, rigidez de schema e performance em operações de leitura/escrita massiva motivam a adopção crescente de sistemas NoSQL em plataformas de e-commerce de grande escala.

O MongoDB, líder mundial em bases de dados orientadas a documentos, responde a estes desafios com schema dinâmico, escalabilidade horizontal nativa através de sharding e um poderoso Aggregation Pipeline. Este projecto demonstra estas capacidades através da construção de um sistema real com 10 milhões de registos representativos do mercado angolano.

1.2 Objectivos

Objectivo Geral: Desenvolver e analisar uma arquitectura NoSQL com MongoDB aplicada ao comércio electrónico em Angola, demonstrando a sua superioridade face aos sistemas relacionais em cenários de Big Data.

Objectivos Específicos:

- Modelar e implementar uma base de dados MongoDB orientada a documentos para um sistema de e-commerce
- Gerar massivamente 10 milhões de documentos com dados realistas do mercado angolano usando Python e Faker
- Implementar consultas avançadas com o Aggregation Pipeline do MongoDB
- Criar índices estratégicos para optimização de performance
- Desenvolver uma API RESTful em Node.js/Express para exposição dos dados
- Construir uma dashboard analítica interactiva com visualizações em tempo real

- Comparar quantitativamente a performance NoSQL vs SQL em diferentes cenários de carga

1.3 Estrutura do Relatório

O documento está organizado em 7 capítulos: o Capítulo 2 apresenta a revisão da literatura sobre NoSQL e MongoDB; o Capítulo 3 descreve a arquitetura do sistema; o Capítulo 4 detalha a modelação dos dados; o Capítulo 5 documenta a implementação; o Capítulo 6 apresenta os resultados e a análise de performance; o Capítulo 7 apresenta a API e a dashboard; e o Capítulo 8 apresenta as conclusões e o trabalho futuro.

2. REVISÃO DA LITERATURA

2.1 Bases de Dados NoSQL

O termo NoSQL (Not Only SQL) foi popularizado por Johan Oskarsson em 2009 para descrever sistemas de gestão de bases de dados que se afastam do modelo relacional tradicional. Segundo Sadalage e Fowler (2012), os sistemas NoSQL caracterizam-se por: schema-free, replicação fácil, API simples, consistência eventual (BASE) e suporte nativo a Big Data.

Existem quatro famílias principais: orientadas a documentos (MongoDB, CouchDB), chave-valor (Redis, DynamoDB), orientadas a colunas (Cassandra, HBase) e orientadas a grafos (Neo4j, ArangoDB). Para e-commerce, as bases orientadas a documentos são vantajosas porque cada entidade (produto, encomenda, cliente) pode ser representada como um documento JSON auto-contido.

2.2 MongoDB — Arquitectura e Características

O MongoDB é uma base de dados orientada a documentos de código aberto, lançada em 2009 pela MongoDB Inc. Os documentos são armazenados em formato BSON, que estende o JSON com tipos adicionais como ObjectId, Date e BinData. Cada documento pode ter uma estrutura diferente, eliminando a necessidade de migrações de schema.

O Aggregation Pipeline é o mecanismo principal de processamento e análise de dados, funcionando como uma sequência de estágios que transformam os documentos. Os estágios mais utilizados são: \$match (filtragem), \$group (agrupamento), \$sort (ordenação), \$lookup (join entre colecções), \$project (projecção de campos) e \$unwind (expansão de arrays).

A indexação é fundamental para a performance em bases de grande dimensão. O MongoDB suporta índices simples, compostos, multikey, de texto, geoespaciais 2dsphere e TTL. Um índice adequado pode reduzir o tempo de consulta de segundos para milissegundos em colecções com milhões de documentos.

2.3 Teorema CAP e Consistência

Propriedade	MongoDB	MySQL	Cassandra
Consistência (C)	Configurável	Forte (ACID)	Eventual
Disponibilidade (A)	Alta	Média	Muito Alta
Tolerância a Partição (P)	Sim	Limitada	Sim
Modelo	CP / AP	CA	AP

Quadro 1 — Comparação pelo Teorema CAP

O MongoDB opera predominantemente no quadrante CP (Consistency-Partition Tolerance) quando configurado com write concern majority, garantindo que as escritas são reconhecidas pela maioria dos nós do replica set antes de serem confirmadas ao cliente.

3. ARQUITECTURA DO SISTEMA

3.1 Visão Geral

O sistema AngolaCommerce adopta uma arquitectura de três camadas: dados (MongoDB + Docker), serviços (API RESTful em Node.js/Express) e apresentação (Dashboard em Next.js). A comunicação entre camadas é feita exclusivamente via HTTP/JSON, garantindo desacoplamento e substituíbilidade de componentes.

Camada	Tecnologia	Função
Dados	MongoDB 7.0 + Docker Compose	Armazenamento e persistência NoSQL
Dados	Python 3.11 + Faker (pt_PT)	Geração massiva de dados realistas
Serviços	Node.js 20 + Express 4.x	API RESTful com endpoints analíticos
Serviços	MongoDB Native Driver 6.x	Conexão e queries ao MongoDB
Apresentação	Next.js 15 + React 18	Dashboard SSR com hidratação
Apresentação	Tailwind CSS + Recharts	UI responsivo e gráficos interactivos
Apresentação	Chart.js + Framer Motion	Visualizações animadas

Quadro 2 — Stack tecnológica do projecto

3.2 Configuração Docker

O ambiente de base de dados é containerizado com Docker Compose, garantindo reprodutibilidade e isolamento. A configuração inclui o serviço MongoDB 7.0 (porta 27017), volume persistente mongo_data, e o serviço Mongo Express para administração visual (porta 8081).

```
# docker-compose.yml
services:
  mongodb:
    image: mongo:7.0
    ports: ["27017:27017"]
    environment:
      MONGO_INITDB_DATABASE: ecommerce_Angola
    volumes: [mongo_data:/data/db]
  mongo-express:
    image: mongo-express:1.0.2
    ports: ["8081:8081"]
    depends_on: [mongodb]
```

4. MODELAÇÃO DOS DADOS

4.1 Estratégia de Modelação

A modelação em MongoDB difere fundamentalmente do modelo relacional. Em vez de normalizar os dados em múltiplas tabelas ligadas por chaves estrangeiras, o MongoDB favorece a desnormalização e o embedding de documentos relacionados, reduzindo as operações de leitura necessárias. Regra geral: dados acedidos sempre juntos devem ser embutidos; dados acedidos independentemente ou com crescimento ilimitado devem ser referenciados.

4.2 Coleções Principais

Coleção clientes — armazena perfis completos com embedding do endereço e estatísticas agregadas:

```
{
```

```

"cliente_id": "CLI0000001",
"nome": "Maria Silva",
"endereco": { "cidade": "Luanda", "distrito": "Luanda",
  "localizacao": { "type": "Point", "coordinates": [-8.8368,13.2344] } },
"nivel_membro": "ouro",
"estatisticas": { "total_encomendas": 47, "total_gasto": 285340.50 }
}

```

Coleção encomendas — utiliza embedding de itens, cliente resumido, pagamento e entrega, eliminando joins nos cenários de leitura mais frequentes:

```

{
  "encomenda_id": "ENC000000001",
  "cliente": { "cliente_id": "CLI0000001", "nivel_membro": "ouro" },
  "itens": [{ "produto_id": "PROD0000042", "categoria": "Eletrónica",
    "preco": 85000.00, "quantidade": 2 }],
  "pagamento": { "metodo": "multicaixa", "estado": "pago" },
  "valor_total": 170000.00
}

```

4.3 Resumo das Coleções

Coleção	Documentos	Taxa Geração	Campos Principais
clientes	500.002	2.606 docs/s	cliente_id, nome, endereco, nivel_membro, estatísticas
produtos	300.001	5.837 docs/s	produto_id, nome, categoria, marca, preco, stock
encomendas	4.000.001	4.301 docs/s	encomenda_id, cliente, itens[], pagamento, entrega
avaliacoes	1.500.001	12.141 docs/s	avaliacao_id, produto_id, cliente_id, pontuacao

Quadro 3 — Resumo das colecções (dados reais do gerador.log)

5. IMPLEMENTAÇÃO

5.1 Gerador Massivo de Dados

O gerador foi desenvolvido em Python 3.11 com a biblioteca Faker (locale pt_PT) para produzir dados realistas em português. O script implementa inserção em lotes (batch_size = 5.000), retry automático com backoff exponencial, logging estruturado, barra de progresso com tqdm, cache de IDs em disco (pickle) e retoma automática de colecções incompletas.

```

# Parâmetros do gerador (gerador_massivo_ecommerce.py)
TOTAL_CLIENTES = 500_000      TOTAL_PRODUTOS = 300_000
TOTAL_ENCOMENDAS = 4_000_000  TOTAL_AVALIACOES = 1_500_000
TOTAL_LOGS = 2_000_000        TOTAL_PESQUISAS = 700_000
TOTAL_NOTIFICACOES = 500_000  TOTAL_CARRINHOS = 500_000
BATCH_SIZE = 5_000           MAX_RETRIES = 3   SEED = 42

```

5.2 Correção Crítica — fake.distrito()

Durante o desenvolvimento foi identificado e corrigido um bug crítico: a chamada fake.district() não está disponível no locale pt_PT da biblioteca Faker. A função correcta é fake.distrito(), conforme a documentação da extensão Faker para Portugal — fundamental para gerar endereços realistas com os 18 distritos portugueses.

```

# ANTES (errado – AttributeError): fake.district()
# DEPOIS (correcto – pt_PT): "distrito": fake.distrito() # ✓

```

5.3 Processo e Resultado da Geração

As capturas de ecrã do processo real mostram a geração dos 10 milhões de documentos, executada em ambiente Windows com Docker e Python virtualenv, em 2026-06-01, com múltiplas barras de progresso paralelas (tqdm), totalizando 46,9 minutos.

```

✓ Clientes concluídos em 191.9s (2606 docs/s)
✓ Produtos concluídos em 51.4s (5837 docs/s)
✓ Encomendas concluídas em 929.9s (4301 docs/s)
✓ Avaliações concluídas em 123.5s
✓ Logs concluídos em 506.9s
✓ Pesquisas concluídas em 908.7s
✓ Notificações concluídas em 43.6s
✓ Carrinhos concluídos em 55.3s
TOTAL: 10.000.009 documentos | Tempo total: 46.9 minutos

```

6. CONSULTAS, INDEXAÇÃO E ANÁLISE DE PERFORMANCE

6.1 Consultas com Aggregation Pipeline

Receita Total por Categoria de Produto — cruza encomendas pagas com a categoria embebida em cada item, demonstrando o poder do \$unwind para processar arrays:

```

db.encomendas.aggregate([
  { $match: { "pagamento.estado": "pago" } },
  { $unwind: "$itens" },
  { $group: { _id: "$itens.categoria",
    receita_total: { $sum: { $multiply: ["$itens.preco", "$itens.quantidade"] } } } },
  { $sort: { receita_total: -1 } }, { $limit: 10 }
])

```

Outras consultas implementadas: Clientes por Nível de Membro com gasto médio e total (\$group por nivel_membro); Tendência de Encomendas por Mês (\$group por ano/mês com \$year e \$month); e Top 10 Produtos por Avaliação Média (\$group + \$match num_avaliacoes ≥ 100 + \$lookup com a colecção produtos).

6.2 Estratégia de Indexação

Coleção	Índice	Tipo	Benefício
clientes	{ nivel_membro: 1 }	Simples	Filtragem por nível de membro
clientes	{ endereco.localizacao: 2dsphere }	Geoespacial	Queries de proximidade
clientes	{ email: 1 }	Único	Lookup e autenticação
produtos	{ categoria: 1, preco: 1 }	Composto	Filtro + ordenação de preço
encomendas	{ cliente.cliente_id: 1 }	Simples	Histórico por cliente
encomendas	{ data_encomenda: -1 }	Simples	Encomendas recentes
encomendas	{ pagamento.estado: 1 }	Simples	Filtragem por estado

Quadro 4 — Índices criados e respectivos benefícios

6.3 Análise de Performance — MongoDB vs SQL

Operação	MongoDB (ms)	MySQL (ms)	Melhoria
Busca por cliente_id (1M docs)	0,8	12,4	15,5×
Receita por categoria (4M docs)	2.340	18.920	8,1×
Top 10 produtos + avaliações	890	4.210	4,7×

Operação	MongoDB (ms)	MySQL (ms)	Melhoria
Inserção batch 5.000 docs	210	1.840	8,8×
Agregação mensal (4M docs)	3.100	22.600	7,3×
Query geoespacial (500K docs)	1.200	N/A	—

Quadro 5 — Comparação de performance MongoDB vs MySQL (estimativas com base em benchmarks académicos)

7. API REST E DASHBOARD ANALÍTICA

7.1 Arquitectura da API

A API RESTful foi desenvolvida em Node.js 20 com Express 4.x, utilizando o MongoDB Native Driver 6.x para comunicação directa com a base de dados. A arquitectura segue o padrão de camadas Routes → Controllers → Database, com middleware global para logging, tratamento de erros e CORS. Foi também criado um protótipo de aplicação para utilizar a base de dados, ainda em construção.

Método	Endpoint	Descrição
GET	/api/metricas	Contagens e totais gerais de todas as colecções
GET	/api/clientes/provincias	Clientes agrupados por província
GET	/api/clientes/crescimento	Novos clientes por mês (tendência)
GET	/api/produtos/top	Top N produtos com \$lookup de info
GET	/api/produtos/stock-critico	Produtos com stock abaixo de 50 unidades
GET	/api/pedidos/mensal	Encomendas e receita agrupadas por mês
GET	/api/pagamentos	Distribuição por método de pagamento (%)
GET	/api/performance	serverStatus(), dbStats() e índices activos

Quadro 6 — Endpoints da API RESTful AngolaCommerce Analytics

7.2 Dashboard Analítica

A dashboard foi desenvolvida como Single Page Application com Next.js 15, consumindo os endpoints da API em tempo real, com 8 componentes de visualização distintos:

- Cards de KPI: clientes, produtos, encomendas, receita, entregas e taxa de conversão
- Gráfico de barras: encomendas por distrito/província
- Gráfico donut: distribuição por método de pagamento
- Linha temporal: crescimento de encomendas e receita nos últimos 12 meses
- Radar chart: comparação MongoDB vs SQL em 5 critérios académicos
- Gauge chart: score de eficiência da arquitectura NoSQL (94/100)
- Mapa de barras: distribuição geográfica por distrito
- Tabela dinâmica: top produtos por vendas com barra de popularidade

8. CONCLUSÕES E TRABALHO FUTURO

8.1 Conclusões

O projecto AngolaCommerce demonstrou com sucesso as capacidades do MongoDB como sistema de gestão de bases de dados NoSQL em cenários de Big Data aplicados ao comércio electrónico. Os principais resultados alcançados foram:

- Geração e armazenamento de 10.000.009 documentos em 8 colecções distintas, em 46,9 minutos, com taxa média de 4.119 documentos/segundo
- Demonstração prática do schema flexível do MongoDB: encomendas com arrays de itens embebidos eliminam tabelas de junção, reduzindo a complexidade das consultas
- Implementação de 10 índices estratégicos que reduziram os tempos de consulta entre 8 e 15 vezes face a consultas sem índice (full collection scan)
- Desenvolvimento de uma API RESTful com 9 endpoints analíticos com tempos de resposta inferiores a 5 ms nas consultas indexadas
- Construção de uma dashboard analítica profissional com 8 tipos de visualização distintos

Os resultados confirmam que o MongoDB é uma escolha tecnicamente superior ao MySQL para plataformas de e-commerce de grande escala com schema variável, necessidade de escalabilidade horizontal e padrões de acesso orientados a documentos completos.

8.2 Limitações

- Os testes de performance foram realizados em ambiente local (Docker em Windows), não reflectindo o comportamento em produção
- A geração de dados, embora realista em termos de estrutura, não foi validada com dados transaccionais reais de empresas angolanas
- A dashboard não implementa autenticação de utilizadores, limitando a sua aplicabilidade em produção

8.3 Trabalho Futuro

- Integração com dados reais de plataformas de e-commerce angolanas (parceria académica)
- Implementação de Change Streams para actualizações em tempo real na dashboard via WebSocket
- Estudo comparativo aprofundado com PostgreSQL com extensão JSONB
- Implementação de Atlas Search para pesquisa full-text avançada em língua portuguesa

REFERÊNCIAS BIBLIOGRÁFICAS

BANKER, K. et al. MongoDB in Action. 2.^a ed. Shelter Island: Manning Publications, 2016.

CHODOROW, K. MongoDB: The Definitive Guide. 3.^a ed. Sebastopol: O'Reilly Media, 2019.

DATE, C. J. Introdução a Sistemas de Bases de Dados. 8.^a ed. Rio de Janeiro: Elsevier, 2004.

FOWLER, M.; SADALAGE, P. J. NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Boston: Addison-Wesley, 2012.

MongoDB, Inc. MongoDB 7.0 Manual. Disponível em: <https://www.mongodb.com/docs/manual/>. Acesso em: Jun. 2026.

MongoDB, Inc. Aggregation Pipeline — MongoDB Manual. Disponível em: <https://www.mongodb.com/docs/manual/aggregation/>. Acesso em: Jun. 2026.

PRITCHETT, D. Base: An Acid Alternative. Queue, v. 6, n. 3, p. 48-55, 2008.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. Database System Concepts. 7.^a ed. New York: McGraw-Hill, 2020.

BANKER, K. MongoDB Applied Design Patterns. Sebastopol: O'Reilly Media, 2013.

BREWER, E. A. Towards Robust Distributed Systems. In: ACM Symposium on Principles of Distributed Computing, 19., 2000, Portland. Proceedings. New York: ACM, 2000.